React Components & Hooks Report

Campus Placement Management System — React Frontend

Module: campus-placement-react (Vite + React 18.3) **Ported from:** campus-placement-system-dashboard_2.html

Course: 21CSE354T **Date:** 20 September 2026

25	10	4	42	10
COMPONENTS	ROUTES	FORMS	HOOK CALLS	CUSTOM HOOKS

> This report documents the React frontend of the Campus Placement Management System against the unit syllabus. It is organised in the order the topics are taught: the foundation of React and JSX (sections 2–3), components (4), events (5), forms (6), refs (7), styling (8) and routing (9); then the rules of hooks (10), the five built-in hooks used (11), writing custom hooks (12), labelling them with `useDebugValue` (13) and finding and reusing them (14). Every code extract is taken from the project as it stands, and section 15 records the browser verification run.

1. Overview

The application is a routed placement dashboard with nine pages: Login, Profile, Job Postings, Eligibility, Applications, Application Detail, Statistics Dashboard, Admin and a 404 page. It is built with **React 18.3.1**, routed with **React Router 7.18** and bundled by **Vite 5.4**. There is no backend — the seed data lives in `src/data/seed.js` and all state is held in memory, so the project stays focused on React itself.

Item	Value
Framework	React 18.3.1 with ReactDOM 18.3.1
Routing	react-router-dom 7.18.4 (BrowserRouter)
Build tool	Vite 5.4.21 with @vitejs/plugin-react (JSX transform, Fast Refresh)
Language	JSX / ES modules
Styling	Global CSS + 2 CSS Modules + inline style objects
Components	25 — 13 shared, 9 pages, 2 providers, 1 root (24 function, 1 class)
Built-in hooks used	`useState`, `useEffect`, `useContext`, `useMemo`, `useRef` — plus `useDebugValue` for labelling custom hooks
Custom hooks	10, of which 5 carry a `useDebugValue` label
Production bundle	209.11 kB JS (67.59 kB gzipped) + 5.24 kB CSS (1.70 kB gzipped)

2. The Foundation of React

React's central idea is that the interface is a *function of state*. Instead of finding an element and changing it, the component describes what the page should look like for the current data; React compares that description with the previous one and updates only the parts of the real DOM that differ.

2.1 Mounting the application

`createRoot` is the React 18 entry point. `StrictMode` double-invokes components and effects in development to expose impure renders and missing cleanup, and `BrowserRouter` makes the URL available to everything inside it.

```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

2.2 What the React version replaces

The HTML build contained twelve IIFE modules that selected DOM nodes and mutated them. Each has a React counterpart that changes state instead.

Behaviour	HTML build (imperative)	React build (declarative)
Section navigation	`scroll` listener writing `style.background` onto each anchor	`<NavLink>` — the router compares the URL and sets the active class itself
Showing a section	all seven sections in one page, reached by `#hash` links	one route per section; the router mounts only the matching page
Login	`table.innerHTML = studentHTML` string swap	a controlled form with validation, then `navigate('/profile')`
Applying to a job	`appTable.insertRow(-1)` plus a manual duplicate scan	`applyToJob()` updates state; the Applications page re-renders itself
Eligibility filter	loops rows setting `style.opacity`	a criteria form in context; rows get a `row-filtered` class from their data
Admin approve / reject	`parseInt`s a cell and writes the number back	`resolveTask()` updates the task in state
Toast notifications	`createElement` + inline styles + `setTimeout` removal	a queue in state rendered by `ToastHost` through a portal
Animated counters	`requestAnimationFrame` writing `el.textContent`	`useCountUp` keeps the value in state; React writes the text
CSV export	scrapes `th`/`td` out of the rendered table	`useCsvExport` serialises the state array

3. JSX

JSX is the markup-like syntax the components are written in. The Vite React plugin compiles every tag into a `React.createElement` call, so JSX is ordinary JavaScript — which is why expressions, `map` and the ternary

operator all work inside it. The features used in this project are:

JSX feature	Example from the project	File
Embedding an expression with `{ }`	`{applications.length + ' application(s)'}`	ApplicationsPage.jsx
Attributes in camelCase	`className`, `onClick`, `colSpan`, `htmlFor`, `tabIndex`	throughout
Self-closing tags	`<input ... />`, `<Outlet />`	Field.jsx, Layout.jsx
Conditional rendering with `&&`	`{ribbon && <span>{ribbon}</span>}`	Panel.jsx
Conditional rendering with a ternary	`{applied ? 'Applied' : 'Apply'}`	JobRow.jsx
Early return for two whole layouts	the profile renders as a table or as a form	ProfilePage.jsx
Rendering a list with `map` and `key`	`{adminTasks.map((task) => <tr key={task.id}>...)}`	AdminPage.jsx
`<Fragment key>`	groups a category header row with its postings, no wrapper element	JobsPage.jsx
Shorthand fragment `<>...</>`	pages return a TopBar and a Panel side by side	every page
Passing children	`<Panel><table>...</table></Panel>`	every page
Spreading props	`<button {...rest}>`	Button.jsx
An inline style object	`style={{ width: value + '%' }}`	ProgressBar.jsx

A list needs a `key` so React can tell which item is which between renders. Every list in this project is keyed by a record id from the seed data, never by the array index:

```
// src/pages/JobsPage.jsx
{grouped.map(([category, categoryJobs]) => (
  <Fragment key={category}>
    <tr>
      <td colSpan={6}><strong>{category}</strong></td>
    </tr>
    {categoryJobs.map((job) => (
      <JobRow
        key={job.id}
        job={job}
        applied={appliedKeys.has(job.company + '|' + job.role)}
        onApply={handleApply}
      />
    ))}
  </Fragment>
))}
```

4. All About Components of React

The application defines **25 components**: 24 function components and 1 class component. They fall into four groups — shared UI pieces, one page per route, two context providers, and the root.

4.1 Shared components

Component	Props	Responsibility
Layout	—	The frame every route renders inside: sidebar, `<Outlet />`, footer and the toast host.
Sidebar	—	Navigation built from `<NavLink>`, which applies its own active class.
TopBar	`title`, `subtitle`, `onExport`	Page heading, the signed-in user and the export / log-out buttons.
Panel	`title`, `note`, `ribbon`, `children`	The white card every page's content sits in. Styled with a CSS Module.
Button	`variant`, `type`, `children`, … rest	One button primitive replacing the four anchor classes of the HTML build.
Field	`label`, `name`, `error`, `touched`, `hint`, `inline`, `children`	A labelled form control with its hint and validation message. Used by all four forms.
JobRow	`job`, `applied`, `onApply`	One posting row; its button is disabled when already applied to or not eligible.
StatusBadge	`status`	Maps a status string to its colour class.
StatCard	`value`, `caption`, `decimals`, `active`	One statistic tile; runs its own count-up animation.
ProgressBar	`label`, `value`, `active`, `delay`	Branch-wise placement rate bar with an ARIA progressbar role.
ToastHost	—	Renders the toast queue into `document.body` through a portal.
Footer	—	Page footer line.
ErrorBoundary class	`children`	Catches render errors below it and shows a recoverable fallback.

4.2 Page components

Page	Route	What it does
LoginPage	`/login`	Role toggle plus a validated login form; focuses the first field with a ref and navigates to the profile on success.
ProfilePage	`/profile`	Shows the profile as a table, or as an edit form with number, select and textarea inputs.
JobsPage	`/jobs`	Search box, postings grouped by category, eligibility dimming and the apply action.
EligibilityPage	`/eligibility`	The criteria form, with a live preview of how many postings would match.
ApplicationsPage	`/applications`	The applications table; each row links to its own URL.

Page	Route	What it does
ApplicationDetailPage	`/applications/:id`	Reads the id from the URL with `useParams` and shows that one application.
DashboardPage	`/dashboard`	Animated statistics, progress bars and the Ctrl+E export shortcut.
AdminPage	`/admin`	Approve / reject the pending queue, with an optional remark.
NotFoundPage	*	The catch-all page for an unknown address.

4.3 Composition: children and props

Components are combined rather than extended. `Panel` knows nothing about what it wraps — whatever is placed between its tags arrives as the `children` prop:

```
// src/components/Panel.jsx
import styles from './Panel.module.css';

export default function Panel({ title, note, ribbon, children }) {
  return (
    <section className={styles.panel}>
      {ribbon && <span className={styles.ribbon}>{ribbon}</span>}
      <h3 className={styles.heading}>{title}</h3>
      {note && <p className={styles.note}>{note}</p>}
      {children}
    </section>
  );
}
```

4.4 The one class component

Error boundaries are the only React feature with no hook equivalent, so `ErrorBoundary` is written as a class. `getDerivedStateFromError` switches to the fallback during render and `componentDidCatch` logs the component stack.

```
// src/components/ErrorBoundary.jsx
export default class ErrorBoundary extends Component {
  state = { error: null };

  static getDerivedStateFromError(error) {
    return { error };
  }

  componentDidCatch(error, info) {
    console.error('Dashboard render failed:', error, info.componentStack);
  }

  render() {
    if (this.state.error) { /* recoverable fallback UI */ }
    return this.props.children;
  }
}
```

5. Events

React attaches one listener at the root and delivers a *synthetic event* — a wrapper with the same API as the native event but consistent across browsers. Handlers are passed as props in camelCase, and they are functions, not strings.

Event prop	Where it is used	What it does
`onClick`	Button, JobRow, ToastHost, Sidebar links	Role toggle, apply, approve / reject, dismissing a toast.
`onChange`	every input in all four forms, and the job search box	Writes the new value into state — the essential half of a controlled input.
`onSubmit`	the `<form>` element on four pages	Runs validation and, if clean, the page's submit handler.
`onBlur`	every validated input	Marks the field as touched so its error message may appear.
`keydown` (DOM listener)	DashboardPage	Ctrl/Cmd + E exports the CSV. Added in an effect because it is a document-level shortcut, not an element's own event.

5.1 Handler patterns

Three forms appear in the project. The first passes a function by name; the second wraps it in an arrow function so an argument can be supplied; the third reads the value out of the event.

```
// 1. Pass the function itself
<input onChange={form.handleChange} onBlur={form.handleBlur} />

// 2. Arrow function, to pass an argument
<Button variant="green" onClick={() => handleDecision(task, true)}>
  Approve
</Button>

// 3. Read the value from the event object
<input
  value={search}
  onChange={(event) => setSearch(event.target.value)}
/>
```

5.2 Preventing the default action

A form submission reloads the page by default. `preventDefault()` is called once, inside the `useForm` hook, so no page has to remember it:

```
// src/hooks/useForm.js
function handleSubmit(onValid) {
  return (event) => {
    event.preventDefault();            // stop the browser's page reload
    const found = validate ? validate(values) : {};
    setErrors(found);
    ...
  };
}
```

The same applies to the keyboard shortcut, where `preventDefault()` stops the browser's own Ctrl+E binding, and to `<Link>`, where React Router prevents the navigation and updates the URL itself.

6. Forms

All four forms are **controlled**: the value of every input comes from state, and every keystroke goes through a change handler that writes it back. The DOM never holds the truth — state does.

Form	Page	Fields	Validation
Login	`/login`	text, password, checkbox	Required; register number must match `RA` + 13 digits; password at least 6 characters; policy must be accepted
Edit profile	`/profile`	text, select, 3 number fields, textarea	Name and skills required; CGPA within 0–10; backlogs not negative; graduation year 2024–2030
Eligibility criteria	`/eligibility`	select, 3 number fields	CGPA within 0–10; CTC and backlogs not negative
Admin remark	`/admin`	text	If given, at least 4 characters
Job search	`/jobs`	search input	None — filters the table as you type

6.1 The controlled-input cycle

```
user types a character
      |
      v
onChange fires with the synthetic event
      |
      v
handleChange reads event.target.name and .value
      |
      v
setValues({ ...values, [name]: value })      state updates
      |
      v
the component re-renders and the input's
value prop is the new state                  screen and state agree
```

6.2 One change handler for every input type

Checkboxes report `checked` rather than `value`, and number inputs hand back strings. Both are normalised in one place, so the pages never deal with it:

```
// src/hooks/useForm.js
function handleChange(event) {
  const { name, type, value, checked } = event.target;
  const nextValue =
    type === 'checkbox' ? checked : type === 'number' ? Number(value) : value;

  setValues((current) => ({ ...current, [name]: nextValue }));

  // Clear this field's error as soon as the user edits it.
  setErrors((current) => {
    if (!current[name]) return current;
    const next = { ...current };
    delete next[name];
    return next;
  });
}
```

6.3 Validation and the "touched" idea

Each form supplies a `validate` function that returns an object keyed by field name. An error is only *shown* once the field has been touched (blurred, or the form submitted), so the form does not shout at the user before they have typed anything.

```
// src/pages/LoginPage.jsx
function validateLogin(values) {
  const errors = {};

  if (!values.username.trim()) {
    errors.username = 'This field is required';
  } else if (values.role === 'student' && !/^RA\d{13}$/i.test(values.username.trim())) {
    errors.username = 'Register number should look like RA2211026010123';
  }

  if (!values.password) {
    errors.password = 'Password is required';
  } else if (values.password.length < 6) {
    errors.password = 'Password must be at least 6 characters';
  }

  if (!values.agree) {
    errors.agree = 'Please accept the placement policy';
  }

  return errors;
}
```

6.4 A field in the markup

`Field` renders the label, the control and the message together. `htmlFor` matches the input's `id`, so clicking the label focuses the control.

```
// src/pages/EligibilityPage.jsx
<Field
  label="Minimum CTC (LPA)"
  name="minCtc"
  error={form.errors.minCtc}
  touched={form.touched.minCtc}
  hint="Postings below this are dimmed on the Job Postings page"
>
  <input
    id="minCtc" name="minCtc" type="number" min="0" className="input"
    value={form.values.minCtc}
    onChange={form.handleChange}
    onBlur={form.handleBlur}
  />
</Field>
```

Because the criteria live in state, the Eligibility page can show a live count of matching postings while the user is still editing the form — before anything is submitted.

7. Refs

A ref is a box whose `.current` can be changed without causing a re-render. That makes it right for two quite different jobs: holding a DOM node, and holding a value that must survive re-renders but is not displayed.

Ref	File	Holds	Why a ref and not state
`usernameRef`	LoginPage.jsx	The register-number `<input>`	To call `.focus()` on the real DOM node when the page opens.
`headingRef`	DashboardPage.jsx	The panel heading	Moves focus into the page so keyboard users land in the content.
`ref`	useIntersectionObserver.js	The element to watch	The observer needs the DOM node itself.
`frameRef`	useCountUp.js	The `requestAnimationFrame` id	Needed only so cleanup can cancel it; rendering does not depend on it.
`nextId`	ToastContext.jsx	The next toast id	Incrementing it must not trigger a render — as state it would loop forever.
`timers`	ToastContext.jsx	Every pending `setTimeout` handle	Bookkeeping for cleanup, never rendered.

7.1 A ref on a DOM element

The ref is attached with the `ref` attribute and read in an effect — never during render, because the node does not exist yet at that point.

```
// src/pages/LoginPage.jsx
const usernameRef = useRef(null);

// Focus the first field when the page opens, and again when the role changes.
useEffect(() => {
  usernameRef.current?.focus();
}, [role]);

...

<input
  ref={usernameRef}
  id="username"
  name="username"
  value={form.values.username}
  onChange={form.handleChange}
/>
```

7.2 A ref as a value that must not cause a render

```
// src/context/ToastContext.jsx
const nextId = useRef(0);      // counter that survives re-renders
const timers = useRef([]);     // every pending setTimeout handle

function showToast(message, type = 'info') {
  const id = nextId.current;
  nextId.current = id + 1;     // changing a ref does not re-render

  setToasts((current) => [...current, { id, message, type }]);

  const timer = setTimeout(() => dismissToast(id), AUTO_DISMISS_MS);
  timers.current.push(timer);
}
```

The rule of thumb used throughout: if changing the value should change what is on screen, it is state; if it should not, it is a ref.

8. Styling React

Three styling approaches are used, each where it fits best.

Approach	Used for	Files
Global stylesheet	The page frame, tables, buttons, forms and toasts — the rules carried over from the HTML build. Imported once in `App.jsx`.	`src/styles/global.css` (230 lines)
CSS Modules	Components whose class names should not be able to clash with anything else. Vite compiles `.panel` into a unique name such as `_panel_1x9k2`.	`Panel.module.css`, `StatCard.module.css`
Inline style objects	Values computed at runtime, such as a progress bar's width.	`ProgressBar.jsx`, `LoginPage.jsx`
Dynamic className	Switching between existing classes from props or state — button variants, status colours, dimmed rows, the active nav link.	`Button.jsx`, `StatusBadge.jsx`, `JobRow.jsx`, `Sidebar.jsx`

8.1 A CSS Module

The stylesheet is imported as an object; each class is a property. The class names in the built page are generated, so they cannot collide with a global rule of the same name.

```
/* src/components/Panel.module.css */
.panel {
  background: #ffffff;
  border: 1px solid #cccccc;
  padding: 16px 18px;
  position: relative;
}

/* src/components/Panel.jsx */
import styles from './Panel.module.css';

<section className={styles.panel}>
```

8.2 An inline style object

Inline styles in React are objects, not strings: the properties are camelCased and numeric values need their unit. This bar's width is only known at runtime, so it belongs here rather than in a stylesheet.

```
// src/components/ProgressBar.jsx
const fillStyle = {
  width: active ? value + '%' : '0%',
  transitionDelay: delay + 'ms',
};

<div className="bar-inner" style={fillStyle} />
```

8.3 Choosing a class from props

```
// src/components/Button.jsx
const VARIANT_CLASS = {
  primary: 'btn',
  green: 'btn btn-green',
  red: 'btn btn-red',
  outline: 'btn-outline',
  muted: 'btn btn-muted',
};

const classes = [VARIANT_CLASS[variant] || VARIANT_CLASS.primary, className]
  .filter(Boolean)
  .join(' ');
```

9. Routing

Routing is handled by **react-router-dom 7.18**. Each section of the old single page is now its own URL, so pages can be linked to, bookmarked and reached with the browser's back and forward buttons.

9.1 The route table

```
// src/App.jsx
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<Navigate to="/dashboard" replace />} />
    <Route path="login" element={<LoginPage />} />
    <Route path="profile" element={<ProfilePage />} />
    <Route path="jobs" element={<JobsPage />} />
    <Route path="eligibility" element={<EligibilityPage />} />
    <Route path="applications" element={<ApplicationsPage />} />
    <Route path="applications/:id" element={<ApplicationDetailPage />} />
    <Route path="dashboard" element={<DashboardPage />} />
    <Route path="admin" element={<AdminPage />} />
    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
```

Router feature	Where	Purpose
`<BrowserRouter>`	main.jsx	Uses the real URL path (the History API), so addresses look like `/jobs` rather than `#jobs`.
`<Routes>` / `<Route>`	App.jsx	The table above: React Router picks the single best match for the current URL.
Nested routes + `<Outlet />`	App.jsx, Layout.jsx	Every page renders inside `Layout`; `<Outlet />` is the slot the matched page fills, so the sidebar is written once.
`index` route + `<Navigate>`	App.jsx	Redirects `/` to `/dashboard`. `replace` keeps the redirect out of the history.
Dynamic segment `:id`	App.jsx	One route serves every application detail page.
`useParams()`	ApplicationDetailPage.jsx	Reads that `:id` out of the URL.
`useNavigate()`	LoginPage, JobsPage, EligibilityPage, ApplicationDetailPage	Navigation from code: after a successful login, after applying, and `navigate(-1)` for Go Back.
`<NavLink>`	Sidebar.jsx	A link that knows whether it is the current route and sets the active class itself.
`<Link>`	ApplicationsPage, NotFoundPage, ApplicationDetailPage	Navigates without a full page reload.
Catch-all `path="*"`	App.jsx	Renders the 404 page for any unmatched address.

9.2 The active link

`NavLink` passes an `isActive` flag to its `className`, which is the whole of what the HTML build's scroll listener used to do by hand.

```
// src/components/Sidebar.jsx
<NavLink
  to={link.to}
  className={({ isActive }) => (isActive ? 'active' : undefined)}
>
  {link.label}
</NavLink>
```

9.3 A dynamic route

```
// src/pages/ApplicationDetailPage.jsx
const { id } = useParams();
const navigate = useNavigate();
const { applications } = usePlacement();

const application = applications.find((app) => app.id === id);

if (!application) { /* "not found" panel with a link back */ }
```

9.4 Navigating from code

```
// src/pages/LoginPage.jsx
function handleValidSubmit(values) {
  login({ username: values.username.trim(), role: values.role });
  showToast('Signed in as ' + ..., 'success');
  navigate('/profile');              // programmatic navigation
}
```

10. Hooks: the Rules of Hooks

React identifies a hook by the *order* in which it is called, not by its name. The first `useState` in a component is always the first slot, the second is always the second, and so on. Two rules keep that order stable, and both hold everywhere in this project.

Rule 1 — Only call hooks at the top level

Never inside a condition, a loop, a nested function or after an early return. If a hook could be skipped on one render, every hook after it would shift by one slot and read the wrong value.

```
// WRONG - the hook is skipped when the list is empty
if (applications.length > 0) {
  const [selected, setSelected] = useState(null);
}

// RIGHT - the hook always runs; the condition goes inside it
const [selected, setSelected] = useState(null);

// RIGHT - this is how useCountUp does it: the effect always runs and
// decides internally whether there is anything to animate.
useEffect(() => {
  if (!active) return;
  ...
}, [target, duration, active]);
```

The same pattern appears in `ProfilePage`, which returns two completely different layouts. Both `useState` and `useForm` are called *before* the early return, so neither can be skipped:

```
// src/pages/ProfilePage.jsx
const { profile, setProfile } = usePlacement();   // every hook first
const showToast = useToast();
useDocumentTitle('CPMS - Profile');
const [editing, setEditing] = useState(false);
const form = useForm(profile, validateProfile);

if (!editing) {
  return ( /* the read-only table */ );           // early return afterwards
}

return ( /* the edit form */ );
```

Rule 2 — Only call hooks from React functions

That means from a component, or from another hook whose name begins with `use`. Every caller in this project qualifies; plain helpers such as `validateLogin`, `formatDate` and `escapeCell` contain no hooks at all, which is why they can live outside components.

Why the naming convention matters

The `use` prefix is not decoration. It is how the linter and React DevTools know a function is a hook, and therefore how they can check rule 1 for you. A function called `getForm` that called `useState` inside would break silently; calling it `useForm` makes the tooling enforce the rules.

11. The Built-in Hooks

Five built-in hooks are used, across 42 call sites, plus `useDebugValue` which is covered separately in section 13.

Hook	Calls	Purpose in this application
`useState`	15	Every piece of changing data: the signed-in user, the profile, the applications list, the admin queue, the eligibility criteria, form values, errors and touched fields, the search text, the toast queue, the animated counter value and the visibility flag.
`useEffect`	8	Every side effect: focusing an input, setting up the IntersectionObserver, the animation frame loop, the keyboard listener, writing to localStorage, the document title and clearing toast timers.
`useMemo`	6	Values that are derived rather than stored: the two context objects, the eligibility-filtered job list, the eligible count, the search + grouping of postings, and the set of already-applied jobs.
`useRef`	6	Two DOM nodes to focus, the observed element, the animation frame id, the toast id counter and the pending timer list.
`useContext`	2	Reads the placement store and the toast queue — wrapped in `usePlacement()` and `useToastContext()` so no component calls it directly.

11.1 useState

`useState` returns the current value and a setter. Calling the setter tells React the component needs to render again with the new value.

```
// src/context/PlacementContext.jsx - the shared data of the whole app
const [user, setUser] = useState(null);
const [profile, setProfile] = useState(STUDENT_PROFILE);
const [applications, setApplications] = useState(INITIAL_APPLICATIONS);
const [adminTasks, setAdminTasks] = useState(INITIAL_ADMIN_TASKS);
const [criteria, setCriteria] = useState(DEFAULT_CRITERIA);
```

The functional form. When the next value depends on the previous one, the setter is given a function rather than a value. React then passes the latest state in, which is correct even if several updates are queued in the same tick:

```
// src/context/PlacementContext.jsx
setApplications((current) => [
  ...current,
  { id: ..., company: job.company, role: job.role, status: 'Applied' },
]);

// src/context/PlacementContext.jsx - updating one item in a list
setAdminTasks((current) =>
  current.map((task) =>
    task.id === taskId && task.pending > 0
      ? { ...task, pending: task.pending - 1, lastRemark: remark }
      : task
  )
);
```

State is replaced, not edited. Every update above builds a new array or object with the spread operator. React compares by reference, so mutating the old one (`applications.push(...)`) would change the data without React noticing.

The lazy initialiser. Passing a *function* to `useState` makes React call it only on the first render. In `useLocalStorage` that means `localStorage` is read once, not on every keystroke elsewhere in the page:

```
// src/hooks/useLocalStorage.js
const [value, setValue] = useState(() => {
  try {
    const stored = window.localStorage.getItem(key);
    return stored === null ? initialValue : JSON.parse(stored);
  } catch {
    return initialValue;
  }
});
```

11.2 useEffect

An effect is for work that reaches *outside* React: the DOM, a browser API, a timer, storage. It runs after the render is on screen. The dependency array says when to run it again, and the returned function is the cleanup, run before the next effect and on unmount.

Effect	File	Dependencies	Cleanup
Focus the first field	LoginPage.jsx	`[role]`	none needed
Focus the panel heading	DashboardPage.jsx	`[]` — once on mount	none needed
Observe the stats panel	useIntersectionObserver.js	`[threshold]`	`observer.disconnect()`
Run the count-up animation	useCountUp.js	`[target, duration, active]`	`cancelAnimationFrame()`
Ctrl+E keyboard shortcut	DashboardPage.jsx	every render (handler reads fresh state)	`removeEventListener('keydown')`
Mirror state into localStorage	useLocalStorage.js	`[key, value]`	none needed — an idempotent write
Set the document title	useDocumentTitle.js	`[title]`	restores the previous title
Clear pending toast timers	ToastContext.jsx	`[]`	`clearTimeout()` for each

```
// src/pages/DashboardPage.jsx - a listener that is always removed
useEffect(() => {
  function onKeyDown(event) {
    if ((event.ctrlKey || event.metaKey) && event.key.toLowerCase() === 'e') {
      event.preventDefault();
      handleExport();
    }
  }
  document.addEventListener('keydown', onKeyDown);
  return () => document.removeEventListener('keydown', onKeyDown);
});
```

Strict Mode mounts, cleans up and remounts every effect once in development. An effect that forgot its cleanup would immediately show up as a duplicate listener or a doubled animation, which is exactly the point of the check.

11.3 useContext

Context carries data down the tree without passing it through every component in between. Two contexts are used: the placement store and the toast queue. Both are read through a small custom hook that throws a clear error if the provider is missing.

```
// src/context/PlacementContext.jsx
const PlacementContext = createContext(null);

export function PlacementProvider({ children }) {
  ...
  return (
    <PlacementContext.Provider value={value}>
      {children}
    </PlacementContext.Provider>
  );
}

export function usePlacement() {
  const context = useContext(PlacementContext);
  if (!context) {
    throw new Error('usePlacement must be used inside <PlacementProvider>');
  }
  return context;
}
```

Because of context, seven of the nine pages take no props at all — they ask for the data they need themselves.

11.4 useMemo

`useMemo` caches the result of a calculation between renders and recomputes it only when a dependency changes. It is used for two reasons here.

To avoid repeating work. The job list is filtered by the criteria, then by the search text, then grouped by category. Without `useMemo` that whole chain would run again on every unrelated render:

```
// src/pages/JobsPage.jsx
const grouped = useMemo(() => {
  const term = search.trim().toLowerCase();
  const matching = term
    ? jobs.filter((job) =>
        job.company.toLowerCase().includes(term) ||
        job.role.toLowerCase().includes(term) ||
        job.location.toLowerCase().includes(term))
    : jobs;

  const map = new Map();
  matching.forEach((job) => {
    if (!map.has(job.category)) map.set(job.category, []);
    map.get(job.category).push(job);
  });
  return [...map.entries()];
}, [jobs, search]);
```

To keep an object's identity stable. A context value is a new object on every render unless it is memoised, and a new object would make every consumer re-render even when nothing changed:

```
// src/context/PlacementContext.jsx
const value = useMemo(
  () => ({
    user, profile, applications, adminTasks, criteria,
    login, logout, applyToJob, resolveTask, setProfile, setCriteria,
  }),
  [user, profile, applications, adminTasks, criteria]
);
```

11.5 useRef

Covered in detail in section 7 with the rest of the refs topic. In short: `useRef` gives a component a box whose `.current` can change without triggering a render — used here for two DOM nodes to focus, the observed element, an animation frame id, the toast id counter and the list of pending timers.

12. Writing Custom Hooks

A custom hook is simply a function whose name starts with `use` and which calls other hooks. It is the mechanism React provides for reusing *stateful logic* — not markup, which is what components reuse.

Ten custom hooks were written. Each exists because a piece of behaviour was needed on more than one page, or because an effect and its cleanup belonged together in one place.

Custom hook	Built-in hooks inside	Returns	Used by
`useForm`	useState ×3, useDebugValue	values, errors, touched, handleChange, handleBlur, handleSubmit, reset, setValues	LoginPage, ProfilePage, EligibilityPage, AdminPage
`usePlacement`	useContext	The shared store and its actions	7 pages + TopBar
`useToastContext`	useContext	toasts, showToast, dismissToast	ToastHost, useToast
`useToast`	useContext (via above), useDebugValue	`showToast(message, type)`	5 pages + useCsvExport
`useLocalStorage`	useState, useEffect, useDebugValue	`[value, setValue]`	LoginPage
`useEligibleJobs`	useMemo ×2, useDebugValue	`{ jobs, eligibleCount }`	JobsPage, EligibilityPage
`useCountUp`	useState, useRef, useEffect, useDebugValue	The formatted number as a string	StatCard
`useIntersectionObserver`	useRef, useState, useEffect	`[ref, isVisible]`	DashboardPage
`useCsvExport`	useToast	`exportCsv(headers, rows, filename)`	ApplicationsPage, DashboardPage
`useDocumentTitle`	useEffect	nothing	All 9 pages

12.1 useForm — the one that earns its keep most

Four forms share it. Without it, each page would repeat the same state, the same change handler, the same touched bookkeeping and the same `preventDefault`. A page now describes only *what* its fields are and *what* counts as valid.

```
// src/hooks/useForm.js
export function useForm(initialValues, validate) {
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});

  useDebugValue(
    Object.keys(values).length + ' fields, ' +
    Object.keys(errors).length + ' error(s)'
  );

  function handleChange(event) { /* see section 6.2 */ }

  function handleBlur(event) {
    const { name } = event.target;
    setTouched((current) => ({ ...current, [name]: true }));
  }

  function handleSubmit(onValid) {
    return (event) => {
      event.preventDefault();
      const found = validate ? validate(values) : {};
      setErrors(found);
      setTouched(
        Object.keys(values).reduce((all, key) => ({ ...all, [key]: true }), {})
      );
      if (Object.keys(found).length === 0) onValid(values);
    };
  }

  function reset() { setValues(initialValues); setErrors({}); setTouched({}); }

  return { values, errors, touched, handleChange, handleBlur,
           handleSubmit, reset, setValues };
}
```

Each form gets its own independent state. Custom hooks share logic, never the data — two components calling `useForm` are as separate as two components calling `useState`.

12.2 useCountUp — state, ref and effect in one unit

```
// src/hooks/useCountUp.js
const EASE_OUT_CUBIC = (t) => 1 - Math.pow(1 - t, 3);

export function useCountUp(target, { duration = 1200, decimals = 0, active = true } = {}) {
  const [value, setValue] = useState(0);
  const frameRef = useRef(0);

  useDebugValue(active ? 'animating to ' + target : 'waiting');

  useEffect(() => {
    if (!active) return;
    const start = performance.now();

    function step(now) {
      const progress = Math.min((now - start) / duration, 1);
      setValue(target * EASE_OUT_CUBIC(progress));
      if (progress < 1) frameRef.current = requestAnimationFrame(step);
    }

    frameRef.current = requestAnimationFrame(step);
    return () => cancelAnimationFrame(frameRef.current);
  }, [target, duration, active]);

  return decimals > 0 ? value.toFixed(decimals) : String(Math.floor(value));
}
```

12.3 useEligibleJobs — a hook built on useMemo

Both the Jobs page and the Eligibility page need to know which postings match a set of criteria. The Eligibility page calls it with the values currently in its form, which is how it can show a live preview before anything is saved.

```
// src/hooks/useEligibleJobs.js
export function useEligibleJobs(criteria) {
  const jobs = useMemo(() => {
    return JOB_POSTINGS.map((job) => ({
      ...job,
      eligible:
        job.ctc >= criteria.minCtc &&
        (criteria.branch === 'All' || job.branches.includes(criteria.branch)),
    }));
  }, [criteria]);

  const eligibleCount = useMemo(
    () => jobs.filter((job) => job.eligible).length,
    [jobs]
  );

  useDebugValue(eligibleCount + ' of ' + jobs.length + ' eligible');

  return { jobs, eligibleCount };
}
```

12.4 useIntersectionObserver — returning a ref

The hook returns the ref for the caller to attach, so the pattern reads like `useState`: a thing to use, and a value to read.

```
// src/hooks/useIntersectionObserver.js
export function useIntersectionObserver(threshold = 0.2) {
  const ref = useRef(null);
  const [isVisible, setIsVisible] = useState(false);

  useEffect(() => {
    const element = ref.current;
    if (!element) return;
    if (!('IntersectionObserver' in window)) { setIsVisible(true); return; }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          setIsVisible(true);
          observer.disconnect();       // animate once only
        }
      });
    }, { threshold });

    observer.observe(element);
    return () => observer.disconnect();
  }, [threshold]);

  return [ref, isVisible];
}

// src/pages/DashboardPage.jsx
const [statsRef, statsVisible] = useIntersectionObserver(0.2);
<div ref={statsRef}> ... <StatCard active={statsVisible} /> ... </div>
```

13. Labelling Custom Hooks with useDebugValue

`useDebugValue` adds a label to a custom hook in the React DevTools inspector. Without it, a component using `useForm` shows only a list of anonymous `State` entries; with it, the hook announces what it currently holds. It is a development aid only — React ignores it in a production build, and it must be called from inside a custom hook, not from a component.

Hook	Label expression	Shown in DevTools as
`useForm`	`keys(values).length + ' fields, ' + keys(errors).length + ' error(s)'`	`Form: 4 fields, 1 error(s)`
`useCountUp`	`active ? 'animating to ' + target : 'waiting'`	`CountUp: animating to 412`
`useEligibleJobs`	`eligibleCount + ' of ' + jobs.length + ' eligible'`	`EligibleJobs: 4 of 6 eligible`
`useLocalStorage`	`key + ' = ' + JSON.stringify(value)`	`LocalStorage: cpms:role = "student"`
`useToast`	`toasts.length + ' toast(s) on screen'`	`Toast: 2 toast(s) on screen`

```
// src/hooks/useLocalStorage.js
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => { /* lazy read */ });

  useDebugValue(key + ' = ' + JSON.stringify(value));

  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(value));
    } catch {
      /* storage blocked (private window) - the app still works */
    }
  }, [key, value]);

  return [value, setValue];
}
```

DevTools drops the `use` prefix when displaying the label, which is why the entries above read `LocalStorage: …` rather than `useLocalStorage: …`.

13.1 The deferred form

`useDebugValue` takes an optional second argument: a formatting function. React calls it only when the hook is actually being inspected, so an expensive label costs nothing during normal development:

```
// The general form - useful when building the label is costly
useDebugValue(date, (d) => d.toISOString());

// The five hooks in this project use the simple form, because their
// labels are cheap string concatenations.
useDebugValue(eligibleCount + ' of ' + jobs.length + ' eligible');
```

The five labelled hooks are the ones with state worth inspecting. `useDocumentTitle` and `useCsvExport` have no meaningful internal state, so a label would add noise rather than information.

14. Finding and Using Custom Hooks

Two questions decide whether behaviour should become a custom hook, and where that hook should come from.

14.1 When to extract one

The hooks in this project were each extracted for one of three reasons:

Signal	Example from this project
The same stateful logic appeared on more than one page	Four forms repeated the same values / errors / touched trio → `useForm`. Two pages needed the same filtering → `useEligibleJobs`.
An effect and its cleanup belonged together	The observer and its `disconnect()` → `useIntersectionObserver`. The animation frame and its `cancelAnimationFrame` → `useCountUp`.
A component was reaching for context plumbing it should not care about	`useToast()` hides the context entirely: a page that only raises a toast never mentions `useContext`.

14.2 Using a hook someone else wrote

Community collections such as *usehooks-ts*, *react-use* and *usehooks.com* publish ready-made hooks for exactly this kind of behaviour — `useLocalStorage`, `useDebounce`, `useMediaQuery`, `useOnClickOutside`. React Router itself ships `useNavigate` and `useParams`, which this project uses rather than reimplementing.

Three things are worth checking before adopting one:

1. **Does it clean up?** Any hook that subscribes to something should return a cleanup function from its effect.
2. **Are the dependency arrays complete?** A missing dependency is the usual source of a stale value.
3. **Does it obey the rules of hooks?** No conditional hook calls inside it.

`useLocalStorage` here was written by hand rather than installed, because it needed only fifteen lines and the `try` / `catch` around blocked storage is specific to how it is used.

14.3 How a hook is used once it exists

Calling a custom hook looks exactly like calling a built-in one, which is the point of the convention:

```
// src/pages/EligibilityPage.jsx - four custom hooks, one built-in, one router
const { criteria, setCriteria } = usePlacement();        // custom (context)
const showToast = useToast();                            // custom (context)
const navigate = useNavigate();                          // react-router
useDocumentTitle('CPMS - Eligibility');                  // custom (effect)

const form = useForm(criteria, validateCriteria);        // custom (state)
const { eligibleCount } = useEligibleJobs(form.values);  // custom (memo)
```

15. Verification

The production build was served with `vite preview` and driven through the Chrome DevTools Protocol, exercising the routes, the forms, the validation rules and the animations against the real rendered page. All twenty-five checks passed, with no console errors or React warnings.

#	Check	Result
1	Index route `/` redirects to `/dashboard`	PASS
2	`NavLink` marks the current route active	PASS
3	Sidebar navigates to `/login`	PASS
4	`useDocumentTitle` updates the browser tab title	PASS
5	`useRef` focuses the first field on mount	PASS
6	Submitting an empty login form shows all 3 validation errors	PASS
7	Register-number format rule is enforced	PASS
8	A valid login navigates to `/profile`	PASS
9	The signed-in user appears in the top bar	PASS
10	Profile CGPA range (0–10) is validated	PASS
11	A valid profile edit saves into context	PASS
12	Eligibility preview updates live while typing (`useMemo`)	PASS
13	Submitting criteria navigates to `/jobs`	PASS
14	Criteria dim the 4 postings below the CTC floor	PASS
15	Search box filters the postings table	PASS
16	Changing the criteria re-enables eligible postings	PASS
17	Applying navigates to `/applications`	PASS
18	The new application row appears in the table	PASS
19	A success toast is raised on apply	PASS
20	View opens `/applications/:id` (`useParams`)	PASS
21	An unknown address renders the 404 page	PASS
22	Admin Approve decrements the pending count (5 → 4)	PASS
23	The remark is stored with the decision	PASS
24	Stat counters animate to 412 / 61 / 14.2	PASS
25	Progress bars animate to 82% / 74% / 68%	PASS

16. Build output and source size

Artefact	Size	Gzipped
`dist/index.html`	0.42 kB	0.28 kB
`dist/assets/index-*.css`	5.24 kB	1.70 kB
`dist/assets/index-*.js`	209.11 kB	67.59 kB

Area	Files	Lines
`src/pages/`	9	891
`src/components/`	13	279
`src/hooks/`	8	264
`src/context/`	2	151
`src/data/`	1	89
`App.jsx` + `main.jsx`	2	58
Total JSX / JS	**35**	**1,732**
CSS (global + 2 modules)	3	286

17. Summary

The React frontend covers every topic in the unit. The foundation and JSX appear in how the page is described at all; components in the 25 pieces the interface is split into; events and forms in the four controlled, validated forms; refs in the focus management and the values that must not trigger renders; styling in the three approaches used side by side; and routing in the ten routes, the nested layout and the dynamic `/applications/:id` page.

On hooks specifically: the five built-in hooks on the syllabus carry the whole application across 42 call sites — no other built-in hook was needed. Ten custom hooks package the reusable logic, five of them labelled with `useDebugValue` so their state is readable in DevTools. Both rules of hooks hold throughout, which is what lets React match each call to the right piece of state on every render.